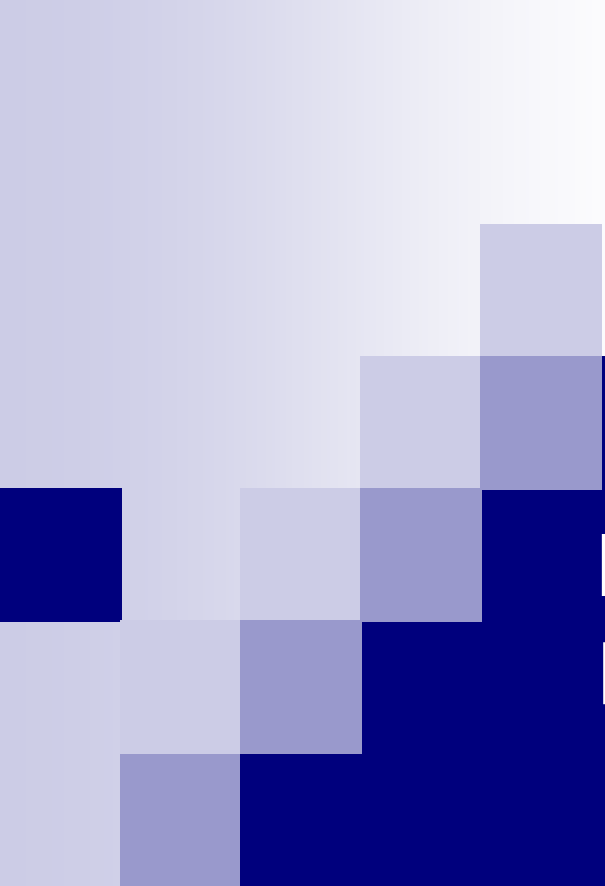




Linguaggi di Programmazione

Docente: Cataldo Musto

Capitolo 2 – Grammatiche e Linguaggi

Si ringraziano il Prof. Giovanni Semeraro, il Prof. Marco de Gemmis e il Prof. Pasquale Lops per la concessione del materiale didattico di base

Definizione di Alfabeto

- Un insieme X finito e non vuoto di simboli è un *alfabeto*.

- **Esempi**
 - L'alfabeto latino, con l'aggiunta dei simboli di interpunzione e dello spazio bianco: $a b c \dots z ; , . :$
 - L'insieme delle dieci cifre arabe: $0 1 \dots 9$
- Con i simboli primitivi dell'alfabeto si formano le parole (es.: $abc, 127, casa, \dots$).

Definizione di Parola o Stringa

- Una sequenza finita di simboli $x_1x_2\dots x_n$, dove ogni x_i è preso da uno stesso alfabeto X è una *parola* (su X).
- Esempio
 - 001110 è una parola su X
 - $X = \{0,1\}$
- **Come si ottiene una parola, dato un alfabeto?**

Definizione di Parola o Stringa

- Una sequenza finita di simboli $x_1x_2\dots x_n$, dove ogni x_i è preso da uno stesso alfabeto X è una **parola** (su X).
- Esempio
 - 001110 è una parola su X
 - $X = \{0,1\}$
- **Come si ottiene una parola, dato un alfabeto?**
 - **Una parola è ottenuta giustappponendo o concatenando simboli (caratteri) dell'alfabeto.**
 - Se una stringa ha m simboli (non necessariamente distinti) allora diciamo che ha lunghezza m .

Lunghezza di una Parola o Stringa

- La **lunghezza** di una stringa w è denotata con $|w|$.
- Le parole di lunghezza 1 sono i simboli di X .
 - Quindi 001110 è una parola di lunghezza 6

$$|001110| = 6$$

Lunghezza di una Parola o Stringa

- La **lunghezza** di una stringa w è denotata con $|w|$.
- Le parole di lunghezza 1 sono i simboli di X .
 - Quindi 001110 è una parola di lunghezza 6

$$|001110| = 6$$

- La parola vuota (o stringa vuota), denotata con λ , è una stringa priva di simboli ed ha lunghezza 0

$$|\lambda| = 0$$

- **Uguaglianza tra stringhe**
 - Due stringhe sono *uguali* se i loro caratteri, letti ordinatamente da sinistra a destra, coincidono.

Definizioni

- X^*
 - L'insieme di tutte le stringhe di lunghezza finita sull'alfabeto X si denota con X^* .

- Esempio
 - Se $X = \{0,1\}$, allora $X^* =$

Definizioni

■ X^*

- L'insieme di tutte le stringhe di lunghezza finita sull'alfabeto X si denota con X^* .

■ Esempio

- Se $X = \{0,1\}$, allora $X^* = \{\lambda, 0, 1, 00, 01, 10, 11, \dots\}$

Importante: la parola vuota appartiene sempre a X^*

Definizioni

■ X^*

- L'insieme di tutte le stringhe di lunghezza finita sull'alfabeto X si denota con X^* .

■ Esempio

- Se $X = \{0,1\}$, allora $X^* = \{\lambda, 0, 1, 00, 01, 10, 11, \dots\}$

IMPORTANTISSIMO:

Non fate confusione tra X (insieme di simboli) e X^* (insieme di parole).

Nel primo sono contenuti i **simboli** dell'alfabeto, nel secondo sono contenute **le parole** dell'alfabeto (ricavabili ovviamente a partire dai simboli)

Definizioni

■ X^*

- L'insieme di tutte le stringhe di lunghezza finita sull'alfabeto X si denota con X^* .

■ Esempio

- Se $X = \{0,1\}$, allora $X^* = \{\lambda, 0, 1, 00, 01, 10, 11, \dots\}$

■ X^* ha un numero di elementi che è un **infinito numerabile**.

- **Cioè è possibile elencarli.**
- Dalla definizione, segue che $\lambda \in X^*$, per ogni insieme X .

Definizioni

■ Concatenazione o prodotto

- Sia $\alpha \in X^*$ una stringa di lunghezza m e $\beta \in X^*$ una stringa di lunghezza n , la **concatenazione** di α e β , denotata con $\alpha\beta$ o $\alpha \cdot \beta$, è definita come la stringa di lunghezza $m+n$, i cui primi m simboli costituiscono una stringa uguale a α ed i cui ultimi n simboli costituiscono una stringa uguale a β .

Definizioni

■ Concatenazione o prodotto

- Sia $\alpha \in X^*$ una stringa di lunghezza m e $\beta \in X^*$ una stringa di lunghezza n , la **concatenazione** di α e β , denotata con $\alpha\beta$ o $\alpha \cdot \beta$, è definita come la stringa di lunghezza $m+n$, i cui primi m simboli costituiscono una stringa uguale a α ed i cui ultimi n simboli costituiscono una stringa uguale a β .
- Quindi se $\alpha = x_1x_2\dots x_m$ e $\beta = x'_1x'_2\dots x'_n$ si ha:

Definizioni

■ Concatenazione o prodotto

- Sia $\alpha \in X^*$ una stringa di lunghezza m e $\beta \in X^*$ una stringa di lunghezza n , la **concatenazione** di α e β , denotata con $\alpha\beta$ o $\alpha \cdot \beta$, è definita come la stringa di lunghezza $m+n$, i cui primi m simboli costituiscono una stringa uguale a α ed i cui ultimi n simboli costituiscono una stringa uguale a β .
- Quindi se $\alpha = x_1x_2\dots x_m$ e $\beta = x'_1x'_2\dots x'_n$ si ha:

$$\alpha\beta = x_1x_2\dots x_mx'_1x'_2\dots x'_n$$

- Se X = alfabeto latino

$$\alpha = \text{capo} \quad \beta = \text{stazione} \quad \alpha\beta = \text{capostazione}$$

Operazione di concatenazione

- **La concatenazione** di stringhe su X è una operazione binaria su X^* :

$$\cdot : X^* \times X^* \rightarrow X^*$$

Significa che prende in **input** due stringhe (prodotto cartesiano di X^*) e restituisce come **output** una nuova stringa

Operazione di concatenazione

- **La concatenazione** di stringhe su X è una operazione binaria su X^* :

$$\cdot : X^* \times X^* \rightarrow X^*$$

- è **associativa** ?

Operazione di concatenazione

- **La concatenazione** di stringhe su X è una operazione binaria su X^* :

$$\cdot : X^* \times X^* \rightarrow X^*$$

- è **associativa**: $(\alpha\beta)\gamma = \alpha(\beta\gamma) = \alpha\beta\gamma, \quad \forall \alpha, \beta, \gamma \in X^*$

Operazione di concatenazione

- **La concatenazione** di stringhe su X è una operazione binaria su X^* :

$$\cdot : X^* \times X^* \rightarrow X^*$$

- ☐ è **associativa**: $(\alpha\beta)\gamma = \alpha(\beta\gamma) = \alpha\beta\gamma, \quad \forall \alpha, \beta, \gamma \in X^*$
- ☐ è **commutativa**?

Operazione di concatenazione

- **La concatenazione** di stringhe su X è una operazione binaria su X^* :

$$\cdot : X^* \times X^* \rightarrow X^*$$

- è **associativa**: $(\alpha\beta)\gamma = \alpha(\beta\gamma) = \alpha\beta\gamma, \quad \forall \alpha, \beta, \gamma \in X^*$
- **non è commutativa**: $\exists \alpha, \beta \in X^* : \alpha\beta \neq \beta\alpha$
 - *capostazione $\neq$ stazionecapo*

Operazione di concatenazione

- **La concatenazione** di stringhe su X è una operazione binaria su X^* :

$$\cdot : X^* \times X^* \rightarrow X^*$$

- è **associativa**: $(\alpha\beta)\gamma = \alpha(\beta\gamma) = \alpha\beta\gamma, \quad \forall \alpha, \beta, \gamma \in X^*$
- **non è commutativa**: $\exists \alpha, \beta \in X^* : \alpha\beta \neq \beta\alpha$
 - *capostazione $\neq$ stazionecapo*
- ha **elemento neutro**?

Operazione di concatenazione

- **La concatenazione** di stringhe su X è una operazione binaria su X^* :

$$\cdot : X^* \times X^* \rightarrow X^*$$

- è **associativa**: $(\alpha\beta)\gamma = \alpha(\beta\gamma) = \alpha\beta\gamma, \quad \forall \alpha, \beta, \gamma \in X^*$
- **non è commutativa**: $\exists \alpha, \beta \in X^* : \alpha\beta \neq \beta\alpha$
 - *capostazione $\neq$ stazionecapo*
- ha **elemento neutro** λ : $\lambda\alpha = \alpha\lambda = \alpha, \quad \forall \alpha \in X^*$

Definizioni

■ Prefisso, Suffisso

- Se $\gamma \in X^*$ è della forma $\gamma = \alpha\beta$, ove $\alpha, \beta \in X^*$, allora α è un *prefisso* di γ e β è un *suffisso* di γ .

Definizioni

■ Prefisso, Suffisso

□ Se $\gamma \in X^*$ è della forma $\gamma = \alpha\beta$, ove $\alpha, \beta \in X^*$, allora α è un *prefisso* di γ e β è un *suffisso* di γ .

□ γ è una parola, cioè una sequenza di simboli appartenenti a X

$$\gamma = x_1 x_2 x_3 \cdots x_{n-2} x_{n-1} x_n$$

Definizioni

■ Prefisso, Suffisso

- Se $\gamma \in X^*$ è della forma $\gamma = \alpha\beta$, ove $\alpha, \beta \in X^*$, allora α è un *prefisso* di γ e β è un *suffisso* di γ .
- La definizione ci dice che la parola può essere immaginata come **concatenazione di due segmenti**, una parte iniziale (detta **prefisso**) e una parte finale (detta **suffisso**). Tali parti non hanno una lunghezza predefinita

$$\gamma = x_1 x_2 x_3 \dots x_{n-2} x_{n-1} x_n$$

α β

Definizioni

■ Prefisso, Suffisso

- Se $\gamma \in X^*$ è della forma $\gamma = \alpha\beta$, ove $\alpha, \beta \in X^*$, allora α è un *prefisso* di γ e β è un *suffisso* di γ .

■ Esempio

- Sia $\gamma = 00110$
- **Proviamo a costruire i relativi insiemi.**

Definizioni

■ Prefisso, Suffisso

- Se $\gamma \in X^*$ è della forma $\gamma = \alpha\beta$, ove $\alpha, \beta \in X^*$, allora α è un *prefisso* di γ e β è un *suffisso* di γ .

■ Esempio

- Sia $\gamma = 00110$. Allora:
 $\{\lambda, 0, 00, 001, 0011, \gamma\}$ è l'insieme dei prefissi di γ

Definizioni

■ Prefisso, Suffisso

- Se $\gamma \in X^*$ è della forma $\gamma = \alpha\beta$, ove $\alpha, \beta \in X^*$, allora α è un *prefisso* di γ e β è un *suffisso* di γ .

■ Esempio

- Sia $\gamma = 00110$. Allora:
 $\{\lambda, 0, 00, 001, 0011, \gamma\}$ è l'insieme dei prefissi di γ
 $\{\lambda, 0, 10, 110, 0110, \gamma\}$ è l'insieme dei suffissi di γ

Importante: anche la stringa vuota e l'intera stringa sono dei prefissi.

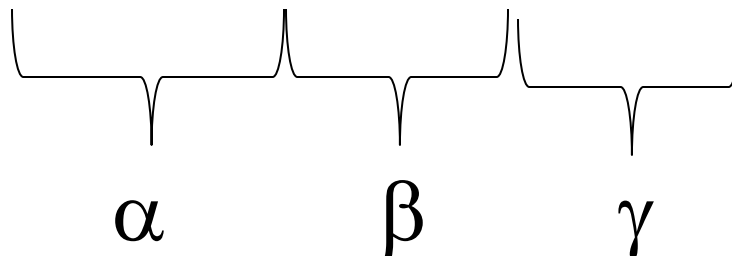
Definizioni

■ Sottostringa

□ Se $\delta, \beta \in X^*$ e δ è della forma $\delta = \alpha\beta\gamma$, ove $\alpha, \beta \in X^*$ allora β è una *sottostringa* di δ .

- In questo caso la definizione estende la precedente immaginando la stringa come concatenazione di **tre** segmenti. Tali segmenti non hanno (così come prima) una lunghezza predefinita. **Quello centrale è detto sottostringa**

$$\delta = x_1 x_2 x_3 \dots x_{n-2} x_{n-1} x_n$$



Definizioni

■ Sottostringa

- Se $\delta, \beta \in X^*$ e δ è della forma $\delta = \alpha\beta\gamma$, ove $\alpha, \gamma \in X^*$ allora β è una *sottostringa* di δ .

■ Esempio

- Sia $\gamma = 00110$. Allora:

Definizioni

■ Sottostringa

- Se $\delta, \beta \in X^*$ e δ è della forma $\delta = \alpha\beta\gamma$, ove $\alpha, \gamma \in X^*$ allora β è una *sottostringa* di δ .

■ Esempio

- Sia $\gamma = 00110$. Allora:

$\{\lambda, 0, 1, 00, 01, 10, 11, 001, 011, 110, 0011, 0110, \gamma\}$

è l'insieme delle sottostringhe di γ .

Definizioni

■ Potenza di una stringa

- Data una stringa α su X , la *potenza h -esima* di α è definita (induttivamente) come segue:

$$\alpha^h = \begin{cases} \lambda & \text{se } h = 0 \\ \alpha\alpha^{h-1} & \text{altrimenti} \end{cases}$$

con $h = 0, 1, 2, \dots$

- La potenza h -esima di una stringa è un caso speciale di concatenamento (in quanto la si ottiene **concatenando una stringa h volte con sè stessa**).

Definizioni

■ Potenza di una stringa

- Data una stringa α su X , la *potenza h -esima* di α è definita (induttivamente) come segue:

$$\alpha^h = \begin{cases} \lambda & \text{se } h = 0 \\ \alpha\alpha^{h-1} & \text{altrimenti} \end{cases}$$

con $h = 0, 1, 2, \dots$

- Esempio

- $\alpha = 010$

Definizioni

■ Potenza di una stringa

- Data una stringa α su X , la *potenza h -esima* di α è definita (induttivamente) come segue:

$$\alpha^h = \begin{cases} \lambda & \text{se } h = 0 \\ \alpha\alpha^{h-1} & \text{altrimenti} \end{cases}$$

con $h = 0, 1, 2, \dots$

- Esempio

- $\alpha = 010$

- $\alpha^2 =$

Definizioni

■ Potenza di una stringa

- Data una stringa α su X , la *potenza h -esima* di α è definita (induttivamente) come segue:

$$\alpha^h = \begin{cases} \lambda & \text{se } h = 0 \\ \alpha\alpha^{h-1} & \text{altrimenti} \end{cases}$$

con $h = 0, 1, 2, \dots$

- Esempio

- $\alpha = 010$
- $\alpha^2 = 010010$

Definizioni

■ Potenza di una stringa

- Data una stringa α su X , la *potenza h -esima* di α è definita (induttivamente) come segue:

$$\alpha^h = \begin{cases} \lambda & \text{se } h = 0 \\ \alpha\alpha^{h-1} & \text{altrimenti} \end{cases}$$

con $h = 0, 1, 2, \dots$

- Esempio

- $\alpha = 010$

- $\alpha^2 = 010010 \quad \alpha^3 = \alpha\alpha^2 =$

Definizioni

■ Potenza di una stringa

- Data una stringa α su X , la *potenza h -esima* di α è definita (induttivamente) come segue:

$$\alpha^h = \begin{cases} \lambda & \text{se } h = 0 \\ \alpha\alpha^{h-1} & \text{altrimenti} \end{cases}$$

con $h = 0, 1, 2, \dots$

- Esempio

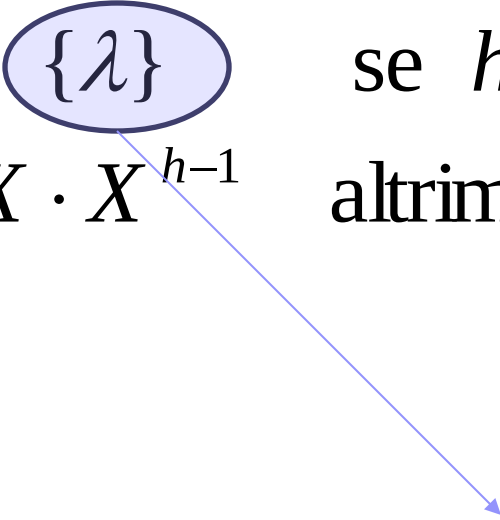
- $\alpha = 010$

- $\alpha^2 = 010010 \quad \alpha^3 = \alpha\alpha^2 = 010010010$
...etc

Definizioni

■ Potenza di un alfabeto

□ Definizione ricorsiva

$$X^h = \begin{cases} \{\lambda\} & \text{se } h = 0 \\ X \cdot X^{h-1} & \text{altrimenti} \end{cases}$$


E' un insieme composto da una stringa.

Non confondiamo **stringa** e **insieme composto da una stringa**

Definizioni

- Potenza di un alfabeto

- Esempio

- $X = \{0,1\}$

- $X^2 =$

- $X^3 =$

$$X^h = \begin{cases} \{\lambda\} & \text{se } h = 0 \\ X \cdot X^{h-1} & \text{altrimenti} \end{cases}$$

Definizioni

- Potenza di un alfabeto

- Esempio

- $X = \{0,1\}$

- $X^2 = X X^{2-1} = X X$

- $X^3 =$

$$X^h = \begin{cases} \{\lambda\} & \text{se } h = 0 \\ X \cdot X^{h-1} & \text{altrimenti} \end{cases}$$

Definizioni

- Potenza di un alfabeto

- Esempio

- $X = \{0,1\}$

- $X^2 = X X = \{0,1\} \{0,1\}$

- $X^3 =$

$$X^h = \begin{cases} \{\lambda\} & \text{se } h = 0 \\ X \cdot X^{h-1} & \text{altrimenti} \end{cases}$$

Definizioni

- Potenza di un alfabeto

- Esempio

- $X = \{0,1\}$

- $X^2 = \{00,01,10,11\}$

- $X^3 =$

$$X^h = \begin{cases} \{\lambda\} & \text{se } h = 0 \\ X \cdot X^{h-1} & \text{altrimenti} \end{cases}$$

Tutte le
possibili
stringhe di
lunghezza 2

Definizioni

- Potenza di un alfabeto

- Esempio

- $X = \{0,1\}$

- $X^2 = \{00,01,10,11\}$

- $X^3 = X X^2 =$

$$X^h = \begin{cases} \{\lambda\} & \text{se } h = 0 \\ X \cdot X^{h-1} & \text{altrimenti} \end{cases}$$

Definizioni

- Potenza di un alfabeto

- Esempio

- $X = \{0,1\}$

- $X^2 = \{00,01,10,11\}$

- $X^3 = X X^2 = \{0,1\} \{00,01,10,11\}$

$$X^h = \begin{cases} \{\lambda\} & \text{se } h = 0 \\ X \cdot X^{h-1} & \text{altrimenti} \end{cases}$$

Definizioni

■ Potenza di un alfabeto

□ Esempio

- $X = \{0,1\}$
- $X^2 = \{00,01,10,11\}$
- $X^3 = \{000,001,010,011,100,101,110,111\}$



Tutte le
possibili
stringhe di
lunghezza 3

$$X^h = \begin{cases} \{\lambda\} & \text{se } h = 0 \\ X \cdot X^{h-1} & \text{altrimenti} \end{cases}$$

Definizioni

■ Potenza di un alfabeto

□ Esempio

- $X = \{0,1\}$
- $X^2 = \{00,01,10,11\}$
- $X^3 = \{000,001,010,011,100,101,110,111\}$



Tutte le
possibili
stringhe di
lunghezza 3

$$X^h = \begin{cases} \{\lambda\} & \text{se } h = 0 \\ X \cdot X^{h-1} & \text{altrimenti} \end{cases}$$

Definizioni

Genericamente, X^k sono tutte le stringhe di lunghezza k , costruibili a partire da un alfabeto X

Unendo tutti i possibili X^k , con $k > 0$ ottengo un nuovo insieme

$$X^+ = X \cup X^2 \cup \dots \cup X^i \cup \dots = \bigcup_{i=1}^{+\infty} X^i$$

Definizioni

Genericamente, X^k sono tutte le stringhe di lunghezza k , costruibili a partire da un alfabeto X

Unendo tutti i possibili X^k , con $k > 0$ ottengo un nuovo insieme

$$X^+ = X \cup X^2 \cup \dots \cup X^i \cup \dots = \bigcup_{i=1}^{+\infty} X^i$$

Insieme di tutte le stringhe di lunghezza maggiore di zero costruibili a partire dall'alfabeto X

Definizioni

Dato X^+ posso facilmente ricavare anche X^*

$$X^+ = X \cup X^2 \cup \dots \cup X^i \cup \dots = \bigcup_{i=1}^{+\infty} X^i$$

$$X^* = \{\lambda\} \cup X^+$$

Insieme di tutte le stringhe di lunghezza maggiore o uguale a zero costruibili a partire dall'alfabeto X

Definizioni

■ Linguaggio formale

- Un *linguaggio formale* L su un alfabeto X è un sottoinsieme di X^* .

$$L \subseteq X^*$$

Definizioni

■ Linguaggio formale

- Un *linguaggio formale* L su un alfabeto X è un sottoinsieme di X^* .

$$L \subseteq X^*$$

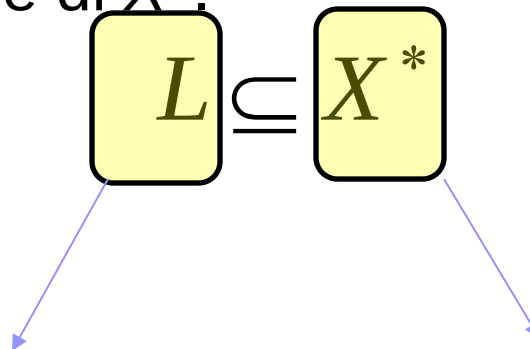


Insieme di tutte le stringhe di lunghezza arbitraria

Definizioni

■ Linguaggio formale

- Un *linguaggio formale* L su un alfabeto X è un sottoinsieme di X^* .



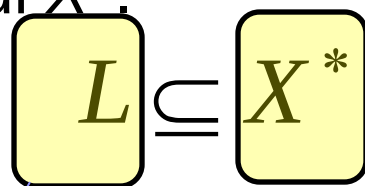
Un **sottoinsieme** di tale insieme

Insieme di tutte le stringhe di lunghezza arbitraria

Definizioni

■ Linguaggio formale

- Un *linguaggio formale* L su un alfabeto X è un sottoinsieme di X^* .



Un **sottoinsieme** di tale insieme

Insieme di tutte le stringhe di lunghezza arbitraria

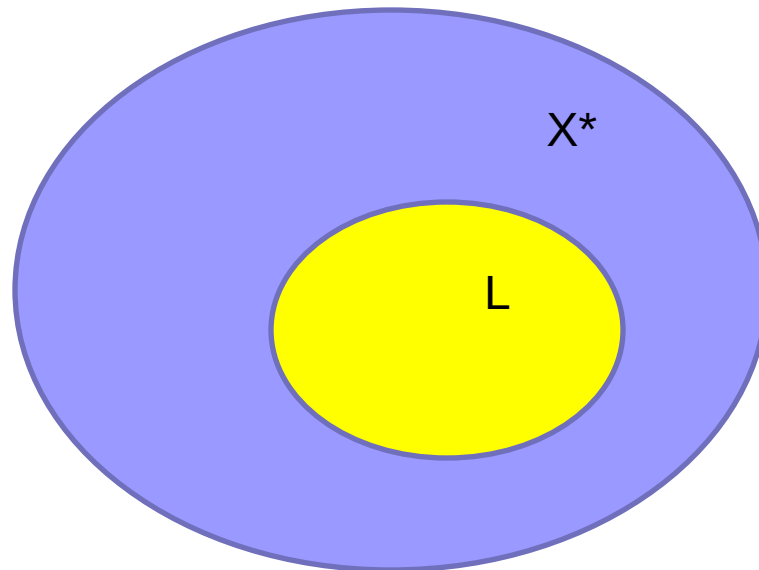
Come è generato? **Intuitivamente**
A partire dalle **regole di una grammatica**

Definizioni

■ Linguaggio formale

- Un *linguaggio formale* L su un alfabeto X è un sottoinsieme di X^* .

$$L \subseteq X^*$$



Definizioni

■ Linguaggio formale

- Un *linguaggio formale* L su un alfabeto X è un sottoinsieme di X^* .

$$L \subseteq X^*$$

■ Esempio:

- Il linguaggio M delle parentesi ben formate è un **linguaggio formale** in quanto, si ha:

$$M \subset \{ (,) \}^*$$

- Ovviamente i linguaggi formali possono essere di natura molto diversa l'uno dall'altro.

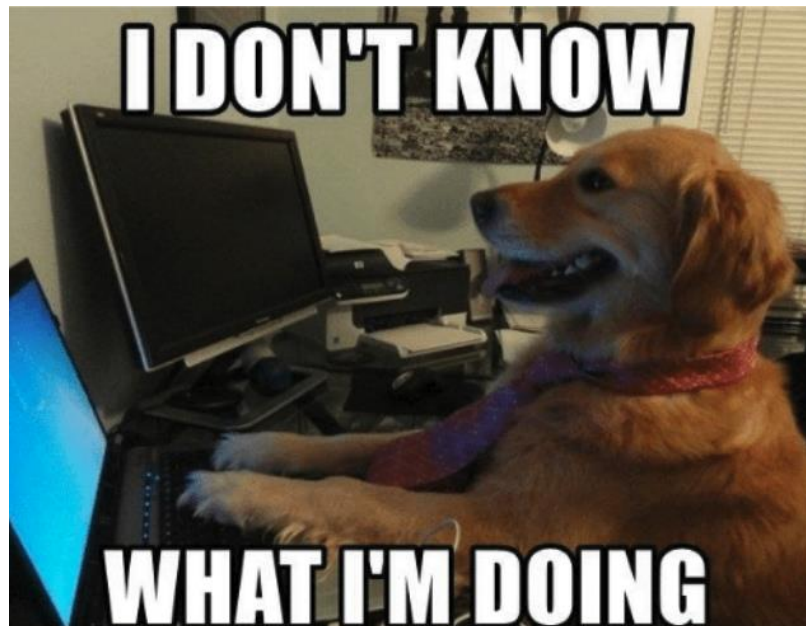
■ Linguaggio formale

- $$L \subseteq X^*$$



Esempi di linguaggi formali

- Fissato X come l'insieme dei simboli presenti sulla tastiera, **un generico codice sorgente in un linguaggio può essere visto come una sequenza di simboli, cioè una stringa (e dunque un elemento di X^*)**



Esempi di linguaggi formali

- Fissato X come l'insieme dei simboli presenti sulla tastiera, **un generico codice sorgente in un linguaggio può essere visto come una sequenza di simboli, cioè una stringa**
 - In questo caso, X^* è **l'insieme di tutte le possibili sequenze di simboli (stringhe) che posso scrivere dentro un editor di testo.**
- L'insieme, finito o infinito, dei programmi **ben costruiti sintatticamente** (ossia, che rispettano la sintassi) costituisce un **linguaggio**.

Problema

- Un linguaggio è un insieme. **Come facciamo a descrivere gli elementi appartenenti a un linguaggio?**

Problema

- Un linguaggio è un insieme. **Come facciamo a descrivere gli elementi appartenenti a un linguaggio?**
 - **Un linguaggio finito** può essere descritto/generato per elencazione degli elementi (se il numero non è troppo grande).
- **Esempi**
 - Il linguaggio dei numeri binari di lunghezza minore o uguale di 4
 - Il linguaggio delle targhe o dei codici fiscali

Problema

- Un linguaggio è un insieme. **Come facciamo a descrivere gli elementi appartenenti a un linguaggio?**
 - **E se il linguaggio è infinito?**
 - Per descrivere gli elementi appartenenti ad un linguaggio infinito, utilizzeremo una notazione insiemistica.
 - Tale notazione si chiama regola di produzione oppure **descrittore del linguaggio**.

Generazione di linguaggi formali

■ Esempio

- Sia L il linguaggio su $X = \{0\}$ costituito dalle stringhe:

$$L = \{\lambda, 00, 0000, 000000, \dots\}$$

Come possiamo esprimere il **descrittore dell'insieme L** ?

Generazione di linguaggi formali

■ Esempio

- Sia L il linguaggio su $X = \{0\}$ costituito dalle stringhe:

$$L = \{\lambda, 00, 0000, 000000, \dots\}$$

Come possiamo esprimere il **descrittore dell'insieme L** ?

$$L = \{w \in X^* \mid \text{numero di zero è pari}\}$$

Generazione di linguaggi formali

■ Esempio

- Sia L il linguaggio su $X = \{0\}$ costituito dalle stringhe:

$$L = \{\lambda, 00, 0000, 000000, \dots\}$$

Come possiamo esprimere il **descrittore dell'insieme L** ?

$$L = \{w \in X^* \mid \text{numero di zero è pari}\}$$



Insieme da cui
attingere le parole



Vincolo che le
caratterizza

Generazione di linguaggi formali

■ Esempio

- Sia L il linguaggio su $X = \{0\}$ costituito dalle stringhe:

$$L = \{\lambda, 00, 0000, 000000, \dots\}$$

Come possiamo esprimere il **descrittore dell'insieme L** ?

$$L = \{w \in X^* \mid \text{numero di zero è pari}\}$$

Tra tutte le
parole di X^*

Il linguaggio è composto solo da quelle che
hanno questa caratteristica

Generazione di linguaggi formali

■ Esempio

- Sia L il linguaggio su $X = \{0\}$ costituito dalle stringhe:

$$L = \{\lambda, 00, 0000, 000000, \dots\}$$

Lo stesso linguaggio può essere descritto con più descrittori,
magari anche un po' più matematici/formali

$$L = \{w \in X^* \mid w=0^n, n=2k, k \geq 0\}$$



Il vincolo è lo stesso, è espresso solo in modo leggermente diverso

Generazione di linguaggi formali

■ Esempio

- Sia L il linguaggio su $X = \{0\}$ costituito dalle stringhe:

$$L = \{\lambda, 00, 0000, 000000, \dots\}$$

Lo stesso linguaggio può essere descritto con più descrittori,
magari anche un po' più matematici/formali

$$L = \{w \in X^* \mid w=0^n, n=2k, k \geq 0\}$$



Abbiamo definito la 'forma' delle parole w , e i valori validi per n (in questo caso quelli pari)

Generazione di linguaggi formali

■ Esempio

- Sia L il linguaggio su $X = \{0\}$ costituito dalle stringhe:

$$L = \{\lambda, 00, 0000, 000000, \dots\}$$

Come possiamo esprimere il **descrittore dell'insieme L** ?

$$L = \{w \in X^* \mid \text{numero di zero è pari}\}$$

$$L = \{w \in X^* \mid w=0^n, n=2k, k \geq 0\}$$

The diagram illustrates the decomposition of the formal language description $L = \{w \in X^* \mid w=0^n, n=2k, k \geq 0\}$ into two components: **struttura** (structure) and **vincoli** (constraints). The expression is enclosed in a large light-blue oval. Two smaller light-blue ovals are positioned within this large oval, one containing $w=0^n$ and the other containing $n=2k, k \geq 0$. A blue arrow points from the first small oval to the word "struttura" below it, and another blue arrow points from the second small oval to the word "vincoli" below it.

Generazione e riconoscimento di linguaggi formali

- Ora che è chiara la definizione di cosa sia un linguaggio, cominciamo a discutere cosa ci interessa studiare dei linguaggi formali
- A noi interessano i linguaggi formali da almeno due *punti di vista*
 - Imparare a **generare** stringhe corrette in un determinato linguaggio
 - Imparare a **riconoscere** (o **far riconoscere!**) in automatico se una stringa è costruita in modo corretto

Generazione e riconoscimento di linguaggi formali

- Ora che è chiara la definizione di cosa sia un linguaggio, cominciamo a discutere cosa ci interessa studiare dei linguaggi formali
- A noi interessano i linguaggi formali da almeno due *punti di vista*
 - Imparare a **generare** stringhe corrette in un determinato linguaggio
 - Punto di vista **descrittivo/generativo**
 - Imparare a **riconoscere** (o far riconoscere!) in automatico se una stringa è costruita in modo corretto
 - Punto di vista **riconoscitivo**

Generazione e riconoscimento di linguaggi formali

■ Punto di vista **Generativo**

- Questa esigenza si riconduce alla costruzione di nuove grammatiche (cioè complessi di regole, come abbiamo visto) che – se applicate nel modo corretto - ci permettono di **generare parole corrette in un certo linguaggio**

Generazione e riconoscimento di linguaggi formali

- Punto di vista **Riconoscitivo**
 - Come possiamo *riconoscere* gli elementi di un dato linguaggio L ?

Generazione e riconoscimento di linguaggi formali

■ Punto di vista **Riconoscitivo**

□ Come possiamo *riconoscere* gli elementi di un dato linguaggio L ?

- Questo secondo punto di vista ha come obiettivo la costruzione di “macchine” in grado di decidere/stabilire **se una stringa** è un elemento di L oppure no.
- Si intende costruire una “macchinetta” cui dare in ingresso una particolare parola e che produce una tra due possibili risposte:

$$sì \equiv ' \in L' \quad e \quad no \equiv ' \notin L'$$

Riconoscimento di linguaggi formali

■ *Punto di vista* Riconoscitivo



Riconoscimento di linguaggi formali

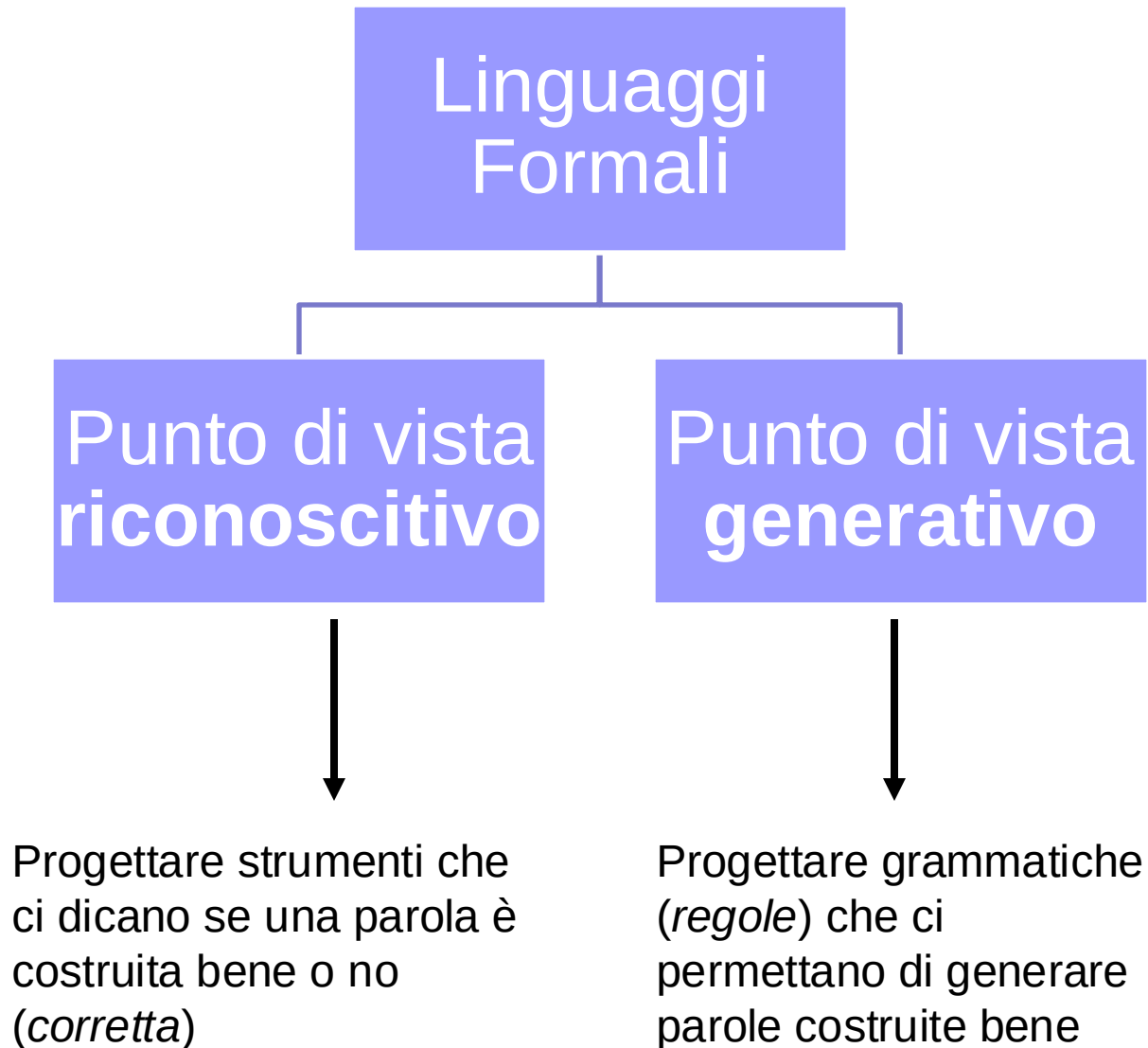
■ *Punto di vista* Riconoscitivo



■ L'esistenza di questa 'macchina' non è sorprendente

- E' equivalente agli analizzatori che prendono in input il vostro codice e vi dicono **se il programma è corretto lessicalmente/sintatticamente oppure no**

Take-Home Message



Generazione di linguaggi formali: esempio

- Sia dato l'alfabeto: $X = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, +, -\}$
Voglio generare il linguaggio L dei numeri interi relativi. Ovviamente: $L \subseteq X^*$
- **Assumiamo, per semplicità, che un numero relativo sia costituito da una serie di cifre precedute da + o -**

Generazione di linguaggi formali: esempio

- Sia dato l'alfabeto: $X = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, +, -\}$
Voglio generare il linguaggio L dei numeri interi relativi. Ovviamente: $L \subseteq X^*$
- **Assumiamo, per semplicità, che un numero relativo sia costituito da una serie di cifre precedute da + o**

Più precisamente, $L \subset X^*$ poiché, ad esempio,

$$1++-5 \notin L$$

Non possiamo elencare gli elementi di L . **Cerchiamo dunque una serie di regole** mediante le quali è possibile generare tutti e soli gli elementi di L .

Generazione di linguaggi formali: esempio

- Cominciamo elencando un po' di numeri, per capirne le caratteristiche essenziali

- ☐
- ☐
- ☐
- ☐
- ☐
- ☐

Generazione di linguaggi formali: esempio

- Cominciamo elencando un po' di numeri, per capirne le caratteristiche essenziali
 - +1
 - -5
 - +204
 - +4888
 - -360
 - -95199

Generazione di linguaggi formali: esempio

- Cominciamo elencando un po' di numeri, per capirne le caratteristiche essenziali

- ☐ +1
- ☐ -5
- ☐ +204
- ☐ +4888
- ☐ -360
- ☐ -95199

Cosa hanno in comune questi numeri?
Possiamo intravedere delle regolarità
nel meccanismo di costruzione?

Ad esempio, **quale è il primo simbolo di ogni stringa?**

Generazione di linguaggi formali: esempio

- Cominciamo elencando un po' di numeri, per capirne le caratteristiche essenziali

- ☐ +1
- ☐ -5
- ☐ +204
- ☐ +4888
- ☐ -360
- ☐ -95199

Cosa hanno in comune questi numeri?
Possiamo intravedere delle regolarità
nel meccanismo di costruzione?

Ad esempio, **quale è il primo simbolo di ogni stringa?**

Ad esempio, ogni numero parte o col più o col meno, seguito da un sequenza di interi

Codifichiamo questo concetto in una regola!

Generazione di linguaggi formali: esempio

- Traduciamo l'intuizione in BNF per descrivere le produzioni:

$$\langle S \rangle ::= + \langle I \rangle$$

$$\langle S \rangle ::= - \langle I \rangle$$

Generazione di linguaggi formali: esempio

- Traduciamo l'intuizione in BNF per descrivere le produzioni:

$$\langle S \rangle ::= + \langle I \rangle$$
$$\langle S \rangle ::= - \langle I \rangle$$

- A questo punto, dobbiamo ragionare su come generare una stringa che descrive un numero. Come si costruisce la parte intera $\langle I \rangle$?

- Ad esempio, quale è la lunghezza?

```
+1  
-5  
+204  
+4888  
-360  
-95199
```

Generazione di linguaggi formali: esempio

- Traduciamo l'intuizione in BNF per descrivere le produzioni:

$$\langle S \rangle ::= + \langle I \rangle$$
$$\langle S \rangle ::= - \langle I \rangle$$

- A questo punto, dobbiamo ragionare su come generare una stringa che descrive un numero. Come si costruisce la parte intera $\langle I \rangle$?

- Un intero può avere una **lunghezza arbitraria**, dunque può essere immaginato come



+1
-5
+204
+4888
-360
-95199

- **Suggerimento:** usiamo la ricorsione

Generazione di linguaggi formali: esempio

- Traduciamo l'intuizione in BNF per descrivere le produzioni:

$$\langle S \rangle ::= + \langle I \rangle$$
$$\langle S \rangle ::= - \langle I \rangle$$

- Un **intero** può avere una lunghezza arbitraria, dunque può essere immaginato come
 - Una **singola** cifra decimale
 - Una **concatenazione** di singole cifre decimali



+1
-5
+204
+4888
-360
-95199

Generazione di linguaggi formali: esempio

- Traduciamo l'intuizione in BNF per descrivere le produzioni:

$$\langle S \rangle ::= + \langle I \rangle$$

$$\langle S \rangle ::= - \langle I \rangle$$

- Un **intero** può avere una lunghezza arbitraria, dunque può essere immaginato come
 - Una **singola** cifra decimale
 - $\langle I \rangle ::= \langle D \rangle$
 - Una **concatenazione** di singole cifre decimali

Generazione di linguaggi formali: esempio

- Traduciamo l'intuizione in BNF per descrivere le produzioni:

$$\langle S \rangle ::= + \langle I \rangle$$

$$\langle S \rangle ::= - \langle I \rangle$$

- Un **intero** può avere una lunghezza arbitraria, dunque può essere immaginato come
 - Una **singola** cifra decimale
 - $\langle I \rangle ::= \langle D \rangle$
 - Una **concatenazione** di singole cifre decimali
 - **Suggerimento:** utilizzare una regola ricorsiva

Generazione di linguaggi formali: esempio

- Traduciamo l'intuizione in BNF per descrivere le produzioni:

$$\langle S \rangle ::= + \langle I \rangle$$

$$\langle S \rangle ::= - \langle I \rangle$$

- Un **intero** può avere una lunghezza arbitraria, dunque può essere immaginato come
 - Una **singola** cifra decimale
 - $\langle I \rangle ::= \langle D \rangle$
 - Una **concatenazione** di singole cifre decimali
 - $\langle I \rangle ::= \langle D \rangle \langle I \rangle$

Generazione di linguaggi formali: esempio

- Traduciamo l'intuizione in BNF per descrivere le produzioni:

$$\langle S \rangle ::= + \langle I \rangle$$

$$\langle S \rangle ::= - \langle I \rangle$$

- **Come si «produce» un numero intero I?**

- Un intero può avere una lunghezza arbitraria, dunque può essere immaginato come una concatenazione di singole cifre decimali

$$\langle I \rangle ::= \langle D \rangle$$

$$\langle I \rangle ::= \langle D \rangle \langle I \rangle$$

Generazione di linguaggi formali: esempio

- Traduciamo l'intuizione in BNF per descrivere le produzioni:

$$\langle S \rangle ::= + \langle I \rangle \mid - \langle I \rangle$$

$$\langle I \rangle ::= \langle D \rangle \mid \langle I \rangle \langle D \rangle$$

**Mancano solo le regole
che generano il lessico**

Generazione di linguaggi formali: esempio

- Traduciamo l'intuizione in BNF per descrivere le produzioni:

$$\langle S \rangle ::= + \langle I \rangle \mid - \langle I \rangle$$

$$\langle I \rangle ::= \langle D \rangle \mid \langle I \rangle \langle D \rangle$$

$$\langle D \rangle ::= 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

Generazione di linguaggi formali: esempio

- Traduciamo l'intuizione in BNF per descrivere le produzioni:

$$\langle S \rangle ::= + \langle I \rangle \mid - \langle I \rangle$$
$$\langle I \rangle ::= \langle D \rangle \mid \langle I \rangle \langle D \rangle$$
$$\langle D \rangle ::= 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

- I è il simbolo nonterminale (da ora in poi, talvolta abbreviato in *NT*), anche detto *categoria sintattica*, che sta ad indicare (e da cui si genera) la classe dei numeri interi.
- I è definito ricorsivamente o come una cifra oppure come un intero seguito da una cifra.
- Ogni intero relativo è generato da queste regole e **niente che non sia un intero relativo può essere generato da queste regole.**

Generazione di linguaggi formali: esempio

- Generazione ad albero:
 - **proviamo a generare l'intero relativo -375**
 - Nella notazione vista per il linguaggio delle parentesi ben formate, tipica per i linguaggi formali, la grammatica diventa:

$$S \rightarrow + I \mid - I$$

$$I \rightarrow D \mid I D$$

$$D \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

Generazione di linguaggi formali: esempio

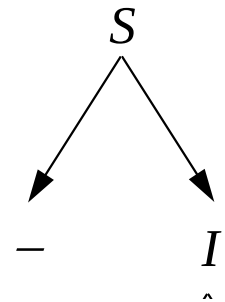
■ Generazione ad albero:

- proviamo a generare l'intero relativo -375
- Nella notazione vista per il linguaggio delle parentesi ben formate, tipica per i linguaggi formali, la grammatica diventa:

$$S \rightarrow + I \mid - I$$

$$I \rightarrow D \mid I D$$

$$D \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$



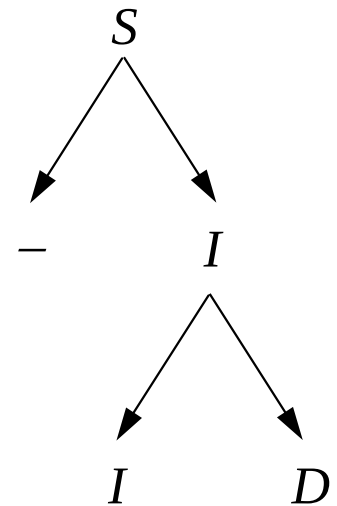
Generazione di linguaggi formali: esempio

- Generazione ad albero:
 - proviamo a generare l'intero relativo -375
 - Nella notazione vista per il linguaggio delle parentesi ben formate, tipica per i linguaggi formali, la grammatica diventa:

$$S \rightarrow + I \mid - I$$

$$I \rightarrow D \mid I D$$

$$D \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$



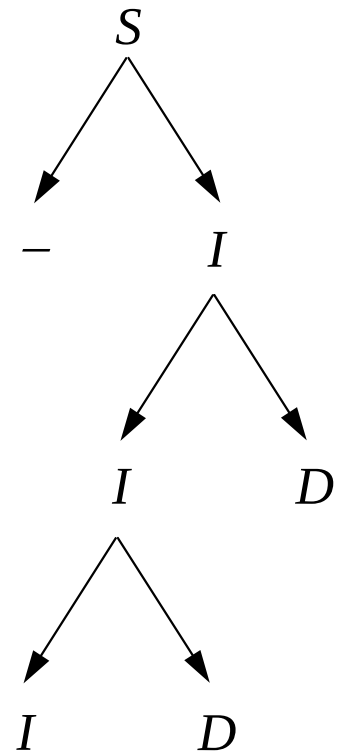
Generazione di linguaggi formali: esempio

- Generazione ad albero:
 - proviamo a generare l'intero relativo -375
 - Nella notazione vista per il linguaggio delle parentesi ben formate, tipica per i linguaggi formali, la grammatica diventa:

$$S \rightarrow + I \mid - I$$

$$I \rightarrow D \mid I D$$

$$D \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$



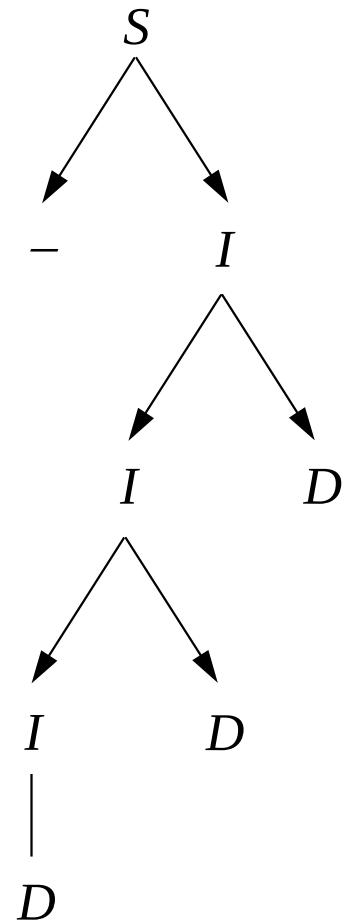
Generazione di linguaggi formali: esempio

- Generazione ad albero:
 - proviamo a generare l'intero relativo -375
 - Nella notazione vista per il linguaggio delle parentesi ben formate, tipica per i linguaggi formali, la grammatica diventa:

$$S \rightarrow + I \mid - I$$

$$I \rightarrow D \mid I D$$

$$D \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$



Generazione di linguaggi formali: esempio

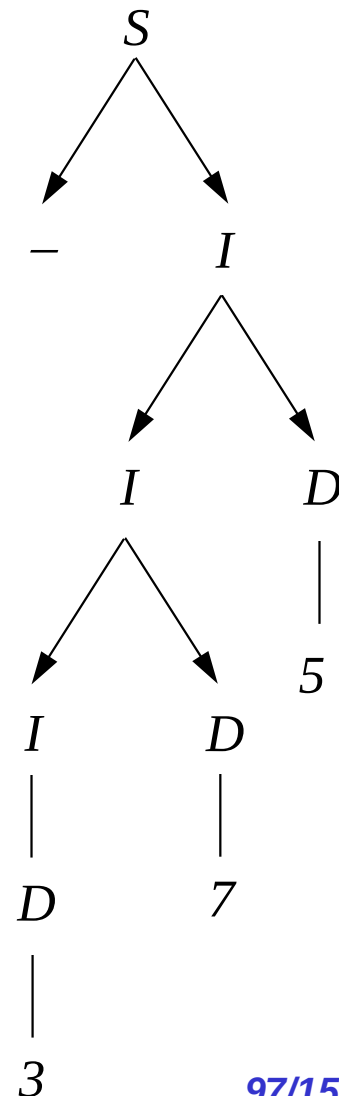
■ Generazione ad albero:

- proviamo a generare l'intero relativo -375
- Nella notazione vista per il linguaggio delle parentesi ben formate, tipica per i linguaggi formali, la grammatica diventa:

$$S \rightarrow + I \mid - I$$

$$I \rightarrow D \mid I D$$

$$D \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$



Generazione di linguaggi formali: esempio

■ Generazione ad albero:

- proviamo a generare l'intero relativo -375
- Nella notazione vista per il linguaggio delle parentesi ben formate, tipica per i linguaggi formali, la grammatica diventa:

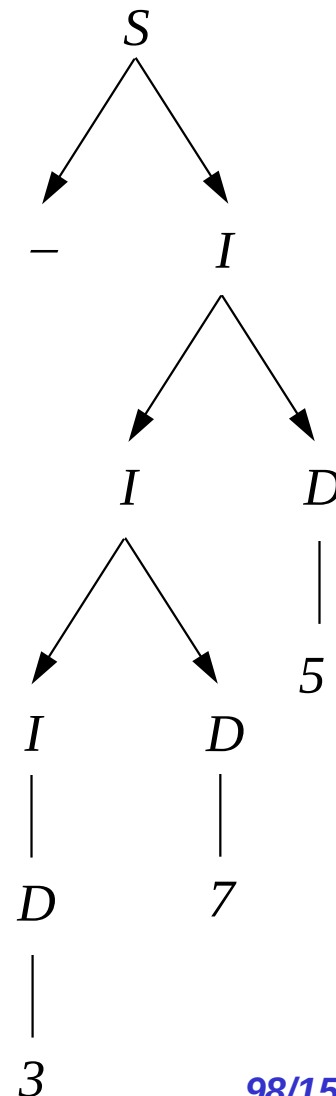
$$S \rightarrow + I \mid - I$$

$$I \rightarrow D \mid I D$$

$$D \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

$$S \overset{*}{\Rightarrow} -375 \iff -375 \in L$$

- Tale albero prende il nome di *albero di derivazione*.



Generazione di linguaggi formali: esempio

- Che problemi ha questa grammatica?

$$S \rightarrow +I \mid -I$$

$$I \rightarrow D \mid ID$$

$$D \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

- Tutti i numeri relativi sono generabili con questa grammatica? **SI**
- Tutto ciò che è generabile da questa grammatica è un numero relativo corretto?

Generazione di linguaggi formali: esempio

- Che problemi ha questa grammatica?

$$S \rightarrow +I \mid -I$$

$$I \rightarrow D \mid ID$$

$$D \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

- Tutti i numeri relativi sono generabili con questa grammatica? **SI**
- Tutto ciò che è generabile da questa grammatica è un numero relativo corretto? **NO**
 - Ad esempio, **019** è una stringa generabile
 - Con piccole modifiche alla grammatica, si può risolvere questo problema. Per esercizio :-)

Grammatiche generative

- Dagli esempi di linguaggi visti abbiamo visto che per generare un linguaggio sono **necessari dei simboli e delle regole**.
Riusciamo a categorizzarli?
- **Prendiamo l'ultima grammatica generata insieme come esempio**. Come classifichiamo i simboli?

$$S \rightarrow + I \mid - I$$

$$I \rightarrow D \mid I D$$

$$D \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

Grammatiche generative

- Dagli esempi di linguaggi visti abbiamo visto che per generare un linguaggio sono **necessari dei simboli e delle regole**.
Riusciamo a categorizzarli?
- **Prendiamo l'ultima grammatica generata insieme come esempio. Come classifichiamo i simboli?**

$$S \rightarrow + I \mid - I$$

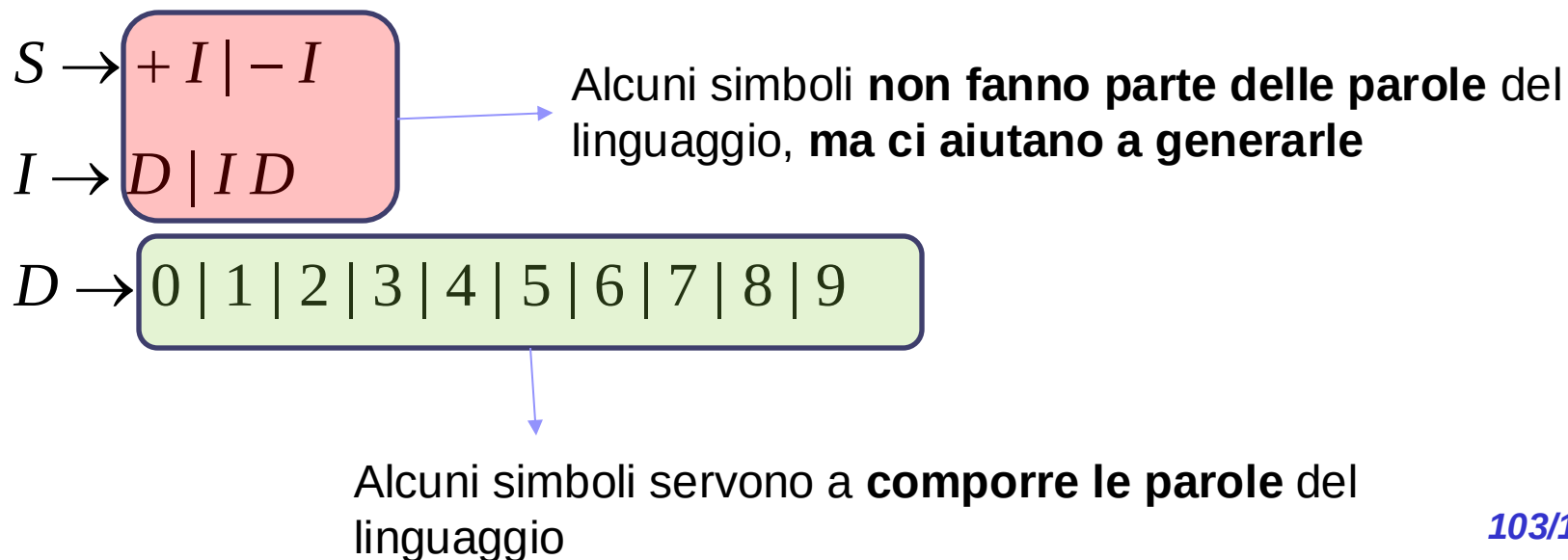
$$I \rightarrow D \mid I D$$

$$D \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

Alcuni simboli servono a **comporre le parole** del linguaggio

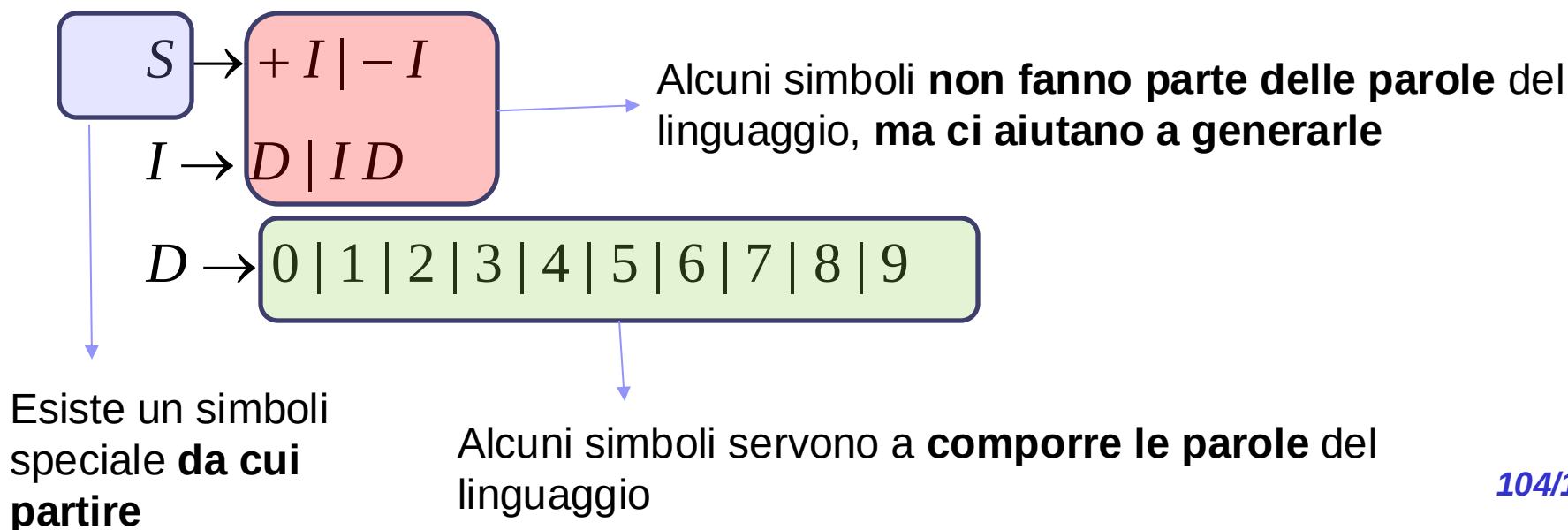
Grammatiche generative

- Dagli esempi di linguaggi visti abbiamo visto che per generare un linguaggio sono **necessari dei simboli e delle regole**.
Riusciamo a categorizzarli?
- **Prendiamo l'ultima grammatica generata insieme come esempio**. Come classifichiamo i simboli?



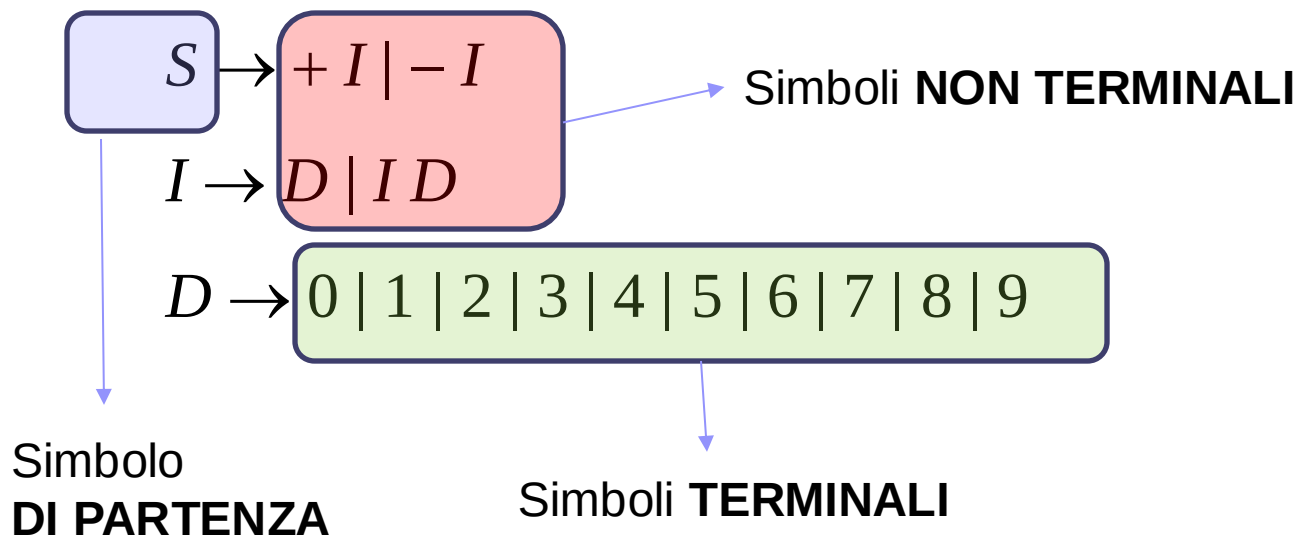
Grammatiche generative

- Dagli esempi di linguaggi visti abbiamo visto che per generare un linguaggio sono **necessari dei simboli e delle regole**.
Riusciamo a categorizzarli?
- **Prendiamo l'ultima grammatica generata insieme come esempio**. Come classifichiamo i simboli?



Grammatiche generative

- Dagli esempi di linguaggi visti abbiamo visto che per generare un linguaggio sono **necessari dei simboli e delle regole**.
Riusciamo a categorizzarli?
- **Prendiamo l'ultima grammatica generata insieme come esempio**. Come classifichiamo i simboli?



Grammatiche generative

- Dagli esempi di linguaggi visti abbiamo visto che per generare un linguaggio sono necessari:
 - **un insieme X di simboli primitivi** con cui si formano le parole del linguaggio, detto *alfabeto dei simboli terminali* o *alfabeto terminale*;
 - **un insieme V di simboli ausiliari o variabili** con cui si identificano le categorie sintattiche del linguaggio, detto *alfabeto dei simboli nonterminali (ausiliari)* o *alfabeto nonterminale* o *alfabeto delle variabili*;
 - **un simbolo speciale S** , scelto tra i nonterminali, da cui far partire la generazione delle parole del linguaggio. Tale simbolo è detto *assioma* o *scopo* o *simbolo distintivo* o *simbolo di partenza* o *simbolo iniziale*;

Grammatiche generative

- Dagli esempi di linguaggi visti abbiamo visto che per generare un linguaggio sono necessari:
 - **un insieme X di simboli primitivi** con cui si formano le parole del linguaggio, detto *alfabeto dei simboli terminali* o *alfabeto terminale*;
 - **un insieme V di simboli ausiliari o variabili** con cui si identificano le categorie sintattiche del linguaggio, detto *alfabeto dei simboli nonterminali (ausiliari)* o *alfabeto nonterminale* o *alfabeto delle variabili*;
 - **un simbolo speciale S** , scelto tra i nonterminali, da cui far partire la generazione delle parole del linguaggio. Tale simbolo è detto *assioma* o *scopo* o *simbolo distintivo* o *simbolo di partenza* o *simbolo iniziale*;
 - **un insieme P di produzioni**, espresse in un formalismo quali regole di riscrittura, BNF ($a ::= b$)

Definizione di Grammatica generativa o a struttura di frase

- Una *grammatica generativa* o *a struttura di frase* G è una quadrupla

$$G = (X, V, S, P)$$

ove:

- X è l'*alfabeto terminale* per la grammatica;
- V è l'*alfabeto nonterminale* o delle variabili per la grammatica;
- S è il *simbolo di partenza* per la grammatica;
- P è l'insieme delle *produzioni* della grammatica ed inoltre valgono le seguenti condizioni:

$$X \cap V = \emptyset \quad \text{e} \quad S \in V$$

Esempi di grammatiche

$$G = (X, V, S, P)$$

- La grammatica per il linguaggio delle parentesi ben formate
- La grammatica per il linguaggio dei numeri interi relativi

Esempi di grammatiche

$$G = (X, V, S, P)$$

- La grammatica per il linguaggio delle parentesi ben formate

$$G_1 = (\{ (,) \}, \{ S \}, S, \{ S \rightarrow (), S \rightarrow (S), S \rightarrow SS \})$$

- La grammatica per il linguaggio dei numeri interi relativi

Esempi di grammatiche

$$G = (X, V, S, P)$$

- La grammatica per il linguaggio delle parentesi ben formate

$$G_1 = (\{ (,) \}, \{ S \}, S, \{ S \rightarrow (), S \rightarrow (S), S \rightarrow SS \})$$

- La grammatica per il linguaggio dei numeri interi relativi

$$G_2 = (\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, +, -\}, \{S, I, D\}, S, \\ \{S \rightarrow +I, S \rightarrow -I, I \rightarrow D, I \rightarrow ID, D \rightarrow 0, D \rightarrow 1, \dots, D \rightarrow 9\})$$

Definizione di Produzione

- Una *produzione* è una coppia (v,w) nella forma $\mathbf{v} \rightarrow \mathbf{w}$,
- Riprendiamo delle produzioni già viste, per capirne meglio la formalizzazione
 - Ovviamente $\mathbf{v}$ è la parte sinistra, $\mathbf{w}$ la parte destra.

$$S \rightarrow + I \mid - I$$

$$I \rightarrow D \mid I D$$

$$D \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

formalizziamone la definizione.

Definizione di Produzione

- Una *produzione* è una coppia (v,w) nella forma $v \rightarrow w$,
ove $v \in (X \cup V)^+$ e $w \in (X \cup V)^*$
e v contiene un NT



Definizione di Produzione

- Una *produzione* è una coppia (v,w) nella forma $\mathbf{v} \rightarrow \mathbf{w}$,
ove $v \in (X \cup V)^+$ e $w \in (X \cup V)^*$
e v contiene un NT
- Riprendiamo le definizioni

$$(X \cup V)^-$$

$$(X \cup V)^+$$

$$(X \cup V)^*$$

Definizione di Produzione

- Una *produzione* è una coppia (v,w) nella forma $v \rightarrow w$,
ove $v \in (X \cup V)^+$ e $w \in (X \cup V)^*$
e v contiene un NT
- Riprendiamo le definizioni

$(X \cup V)$  Insieme di **simboli terminali e non terminali**

$(X \cup V)^+$

$(X \cup V)^*$

Definizione di Produzione

- Una *produzione* è una coppia (v,w) nella forma $v \rightarrow w$,
ove $v \in (X \cup V)^+$ e $w \in (X \cup V)^*$
e v contiene un NT

- Riprendiamo le definizioni

$(X \cup V)^+$  Insieme di **simboli terminali e non terminali**

$(X \cup V)^+$ **Nella grammatica dei numeri, l'insieme è composto da S, D, I, +, - e tutti i numeri da zero a nove.**

$(X \cup V)^*$ **Insieme di SIMBOLI, non STRINGHE**

Definizione di Produzione

- Una *produzione* è una coppia (v,w) nella forma $v \rightarrow w$,
ove $v \in (X \cup V)^+$ e $w \in (X \cup V)^*$
e v contiene un NT

- Riprendiamo le definizioni

$(X \cup V)$  Insieme di **simboli terminali e non terminali**

$(X \cup V)^+$  Insieme di **stringhe composte da simboli terminali e non terminali (escluso lambda)**

$(X \cup V)^*$

Definizione di Produzione

- Una *produzione* è una coppia (v,w) nella forma $v \rightarrow w$,
ove $v \in (X \cup V)^+$ e $w \in (X \cup V)^*$
e v contiene un NT

- Riprendiamo le definizioni

$(X \cup V)$  Insieme di **simboli terminali e non terminali**

$(X \cup V)^+$  Insieme di **stringhe composte da simboli terminali e non terminali (escluso lambda)**

$(X \cup V)^*$  Insieme di **stringhe composte da simboli terminali e non terminali (incluso lambda)**

Definizione di Produzione

- Una *produzione* è una coppia (v,w) nella forma $v \rightarrow w$,
ove $v \in (X \cup V)^+$ e $w \in (X \cup V)^*$
e v contiene un NT
- La definizione dunque ci dice che **a sinistra di una regola**
 - Ho una stringa (anche di lunghezza uno, cioè un simbolo)
 - Ci deve essere un non terminale per forza
 - **Non è sorprendente, perché devo per forza ‘riscrivere’ qualcosa**
 - Non posso avere lambda
- **A destra di una regola** invece
 - Posso avere qualsiasi stringa (anche lambda).

Definizione di derivazione o produzione diretta

- Intuitivamente, una derivazione è il **risultato dell'applicazione di una regola di produzione**
- **Formalizziamolo**

Definizione di derivazione o produzione diretta

- Intuitivamente, una derivazione è il **risultato dell'applicazione di una regola di produzione**
- **Scriveremo $y \Rightarrow z$ se la stringa y si trasforma in z applicando una delle regole della grammatica**

Definizione di derivazione o produzione diretta

- Intuitivamente, una derivazione è il **risultato dell'applicazione di una regola di produzione**
- Scriveremo $y \Rightarrow z$ se la stringa y si trasforma in z applicando una delle regole della grammatica
- Prendiamo la grammatica delle parentesi
 - ☐ $s \rightarrow ()$
 - ☐ $s \rightarrow (s)$
 - ☐ $s \rightarrow ss$
- Posso dire che $s \rightarrow ()$?

Definizione di derivazione o produzione diretta

- Intuitivamente, una derivazione è il **risultato dell'applicazione di una regola di produzione**
- Scriveremo $y \Rightarrow z$ se la stringa y si trasforma in z applicando una delle regole della grammatica
- Prendiamo la grammatica delle parentesi
 - ☐ $s \rightarrow ()$
 - ☐ $s \rightarrow (s)$
 - ☐ $s \rightarrow ss$
- Posso dire che $s \rightarrow ()$? **SI**

Definizione di derivazione o produzione diretta

- Intuitivamente, una derivazione è il **risultato dell'applicazione di una regola di produzione**
- Scriveremo $y \Rightarrow z$ se la stringa y si trasforma in z applicando una delle regole della grammatica
- Prendiamo la grammatica delle parentesi
 - ☐ $s \rightarrow ()$
 - ☐ $s \rightarrow (s)$
 - ☐ $s \rightarrow s s$
- Posso dire che $s \rightarrow (())$? **NO**
 - ☐ **Perché tale regola non esiste**

Definizione di derivazione

Viceversa, scriviamo

$$y \xRightarrow{*} z$$

e diciamo che y *produce* z o che z è *derivabile* da y se $y = z$ o se si può ottenere **mediante una sequenza di applicazione di regole di produzione.**

Definizione di derivazione

Viceversa, scriviamo

$$y \xRightarrow{*} z$$

e diciamo che y **produce** z o che z è **derivabile** da y
se $y = z$ o se si può ottenere **mediante una sequenza
di applicazione di regole di produzione.**

Tornando all'esempio precedente

- Posso dire che $s \xRightarrow{*}))$?

Definizione di derivazione

Viceversa, scriviamo

$$y \xRightarrow{*} z$$

e diciamo che y **produce** z o che z è **derivabile** da y
se $y = z$ o se si può ottenere **mediante una sequenza di applicazione di regole di produzione.**

Tornando all'esempio precedente

■ Posso dire che $s \xRightarrow{*}))$? **SI**

- Perché applicando una sequenza di regole che parte dalla parte sinistra, si riesce ad arrivare alla parte destra

Definizione di derivazione

Viceversa, scriviamo

$$y \xRightarrow{*} z$$

e diciamo che y *produce* z o che z è *derivabile* da y se $y = z$ o se si può ottenere **mediante una sequenza di applicazione di regole di produzione.**

Tornando all'esempio precedente

■ Posso dire che $s \xRightarrow{*} ()$? **SI**

□ $s \rightarrow (s) \rightarrow (())$

Definizione di linguaggio generato da una grammatica

- Sia $G = (X, V, S, P)$ una grammatica. Il *linguaggio generato da G* , denotato con $L(G)$, è l'insieme delle stringhe di terminali derivabili dal simbolo di partenza S .

$$L(G) = \left\{ w \in X^* \mid S \xRightarrow[G]{*} w \right\}$$

- Sono, dunque, stringhe di $L(G)$ le stringhe che:

Definizione di linguaggio generato da una grammatica

- Sia $G = (X, V, S, P)$ una grammatica. Il *linguaggio generato da G* , denotato con $L(G)$, è l'insieme delle stringhe di terminali derivabili dal simbolo di partenza S .

$$L(G) = \left\{ w \in X^* \mid S \xRightarrow[G]{*} w \right\}$$

- Sono, dunque, stringhe di $L(G)$ le stringhe che:
 - possono essere derivate da S in G
 - **Consistono di soli terminali**

Un possibile esercizio

- Sia $G = (X, V, S, P)$, ove

$$X = \{a, b\}, \quad V = \{S\}, \quad P = \left\{ S \xrightarrow{(1)} aSb, S \xrightarrow{(2)} ab \right\}$$

Determiniamo $L(G)$.

Questo esercizio si risolve
**applicando a campione delle
regole** e generando un po' di
stringhe valide, al fine di
ricavare un descrittore

Esempio – Determiniamo $L(G)$

- Sia $G = (X, V, S, P)$, ove

$$X = \{a, b\}, \quad V = \{S\}, \quad P = \left\{ S \xrightarrow{(1)} aSb, S \xrightarrow{(2)} ab \right\}$$

$$ab \in L(G) \quad \text{poiché} \quad S \xRightarrow{(2)} ab$$

Se numeriamo le produzioni, possiamo indicare la produzione usata immediatamente al di sotto del simbolo $\Rightarrow$.

$$\xRightarrow{(n)} \equiv \text{ho applicato la produzione } n$$

Esempio

$$X = \{a, b\}, \quad V = \{S\}, \quad P = \left\{ S \xrightarrow{(1)} aSb, S \xrightarrow{(2)} ab \right\}$$

■ $a^2b^2 \in L(G) \quad ?$

■ $a^3b^3 \in L(G) \quad ?$

Esempio

$$X = \{a, b\}, \quad V = \{S\}, \quad P = \left\{ S \xrightarrow{(1)} aSb, S \xrightarrow{(2)} ab \right\}$$

- $a^2b^2 \in L(G)$ si, poiché $S \xRightarrow{(1)} aSb \xRightarrow{(2)} a^2b^2$
- $a^3b^3 \in L(G)$ si, poiché $S \xRightarrow{3} a^3b^3$
.....

Esempio

$$X = \{a, b\}, \quad V = \{S\}, \quad P = \left\{ S \xrightarrow{(1)} aSb, S \xrightarrow{(2)} ab \right\}$$

■ $a^2b^2 \in L(G)$ si, poiché $S \xRightarrow{(1)} aSb \xRightarrow{(2)} a^2b^2$

■ $a^3b^3 \in L(G)$ si, poiché $S \xRightarrow{3} a^3b^3$

.....

$$\{a^n b^n \mid n > 0\} \subseteq L(G)$$

Esempio

$$X = \{a, b\}, \quad V = \{S\}, \quad P = \left\{ S \xrightarrow{(1)} aSb, S \xrightarrow{(2)} ab \right\}$$

■ $a^2b^2 \in L(G)$ si, poiché $S \xRightarrow{(1)} aSb \xRightarrow{(2)} a^2b^2$

■ $a^3b^3 \in L(G)$ si, poiché $S \xRightarrow{3} a^3b^3$

.....

$$\{a^n b^n \mid n > 0\} \subseteq L(G)$$

- Inoltre, qualsiasi derivazione da S in G produce frasi del tipo $a^n b^n$.

□ Dunque

$$L(G) = \{a^n b^n \mid n > 0\}$$

Esercizio

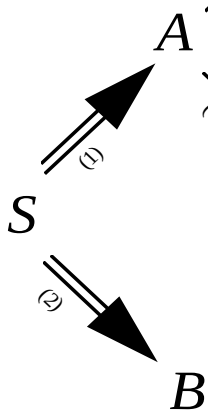
- Sia data la seguente grammatica:

$$S \xrightarrow{(1) (2)} A|B, \quad A \xrightarrow{(3) (4)} aA \mid a, \quad B \xrightarrow{(5) (6)} bB \mid b$$

Determinare $L(G)$.

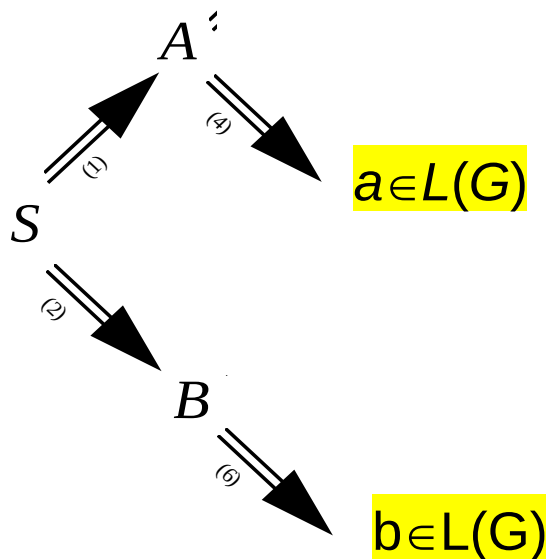
Esercizio

$$S \xrightarrow{(1) \ (2)} A \mid B, \quad A \xrightarrow{(3) \ (4)} aA \mid a, \quad B \xrightarrow{(5) \ (6)} bB \mid b$$



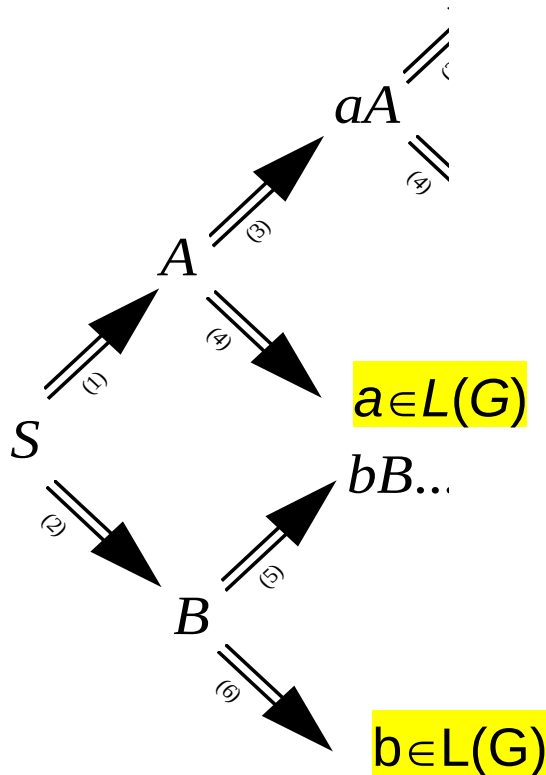
Esercizio

$$S \xrightarrow{(1)} A \mid B, \quad A \xrightarrow{(3)} aA \mid a, \quad B \xrightarrow{(5)} bB \mid b$$



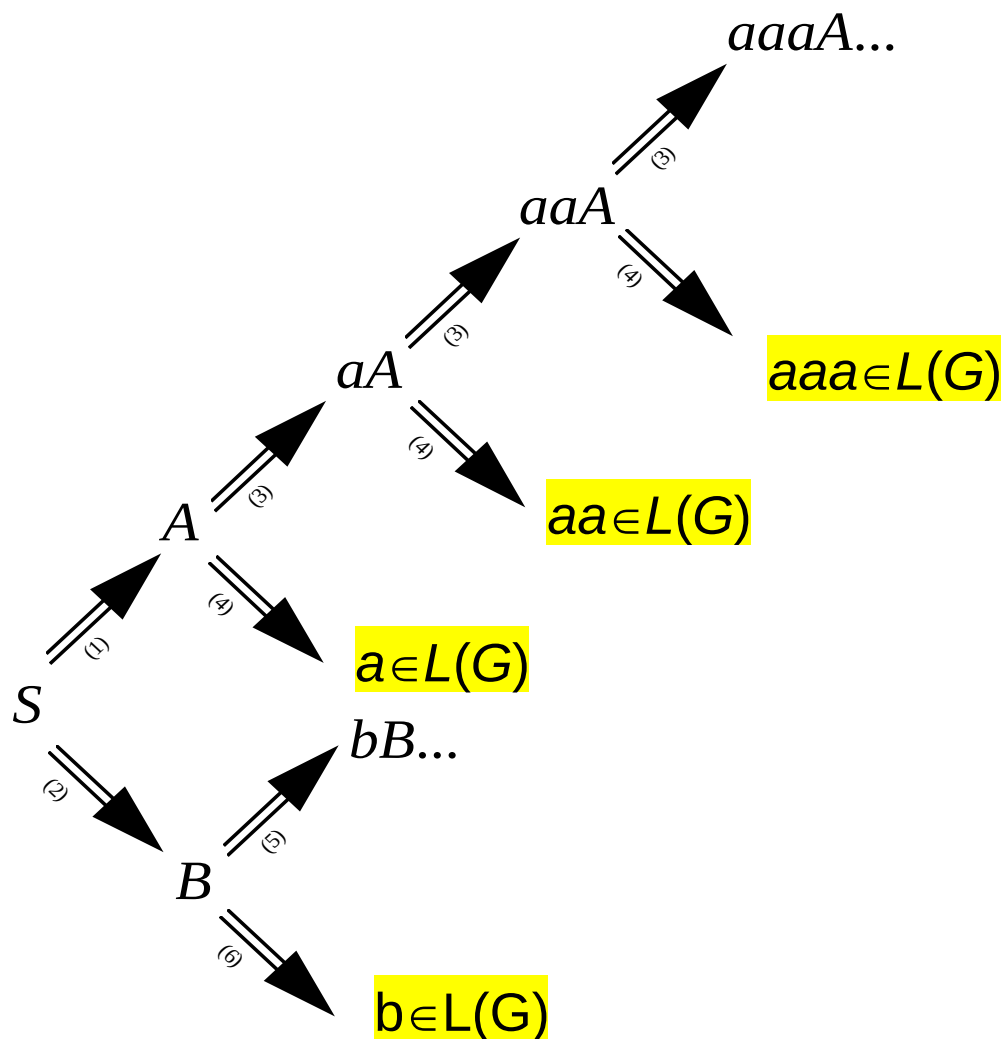
Esercizio

$$S \xrightarrow{(1)} A \mid B, \quad A \xrightarrow{(3)} aA \mid \overset{(4)}{a}, \quad B \xrightarrow{(5)} bB \mid \overset{(6)}{b}$$



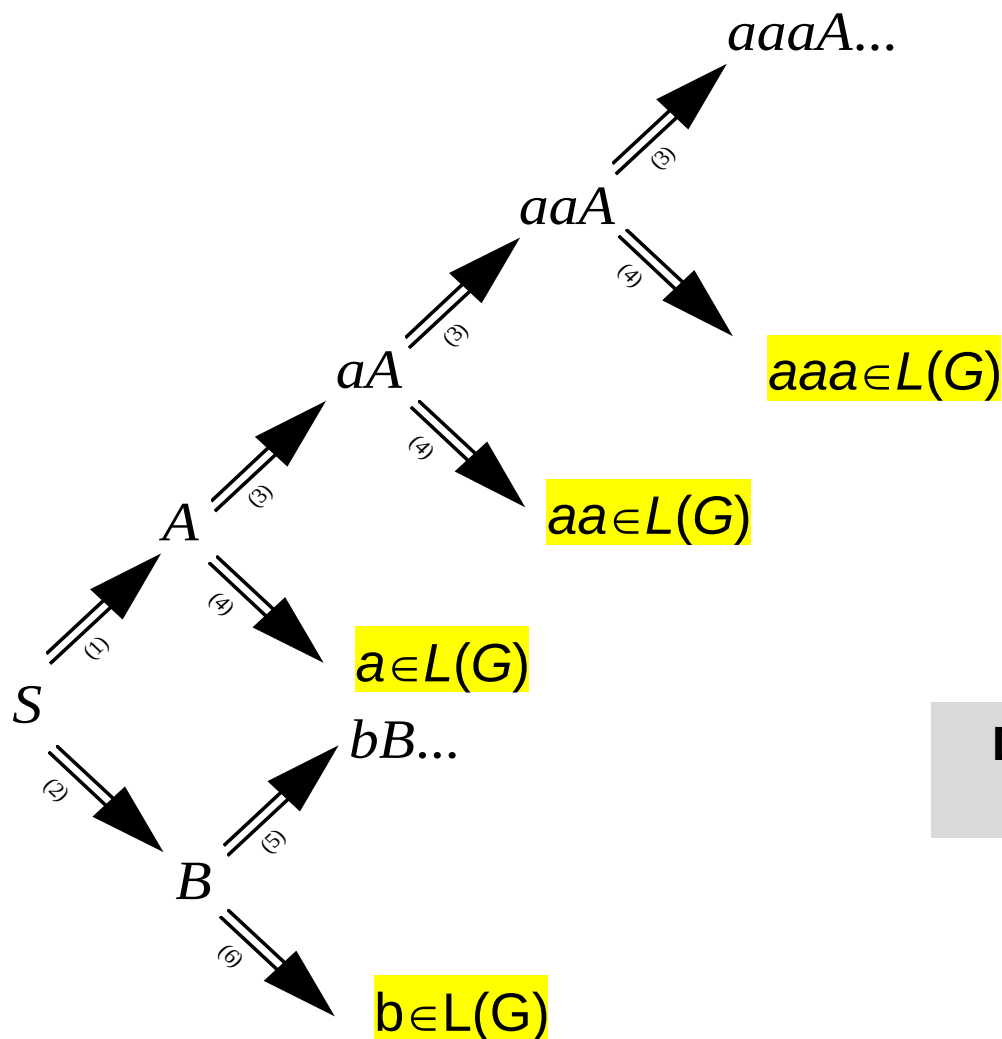
Esempio

$S \xrightarrow{(1)} A \mid B, \quad A \xrightarrow{(3)} aA \mid \xrightarrow{(4)} a, \quad B \xrightarrow{(5)} bB \mid \xrightarrow{(6)} b$



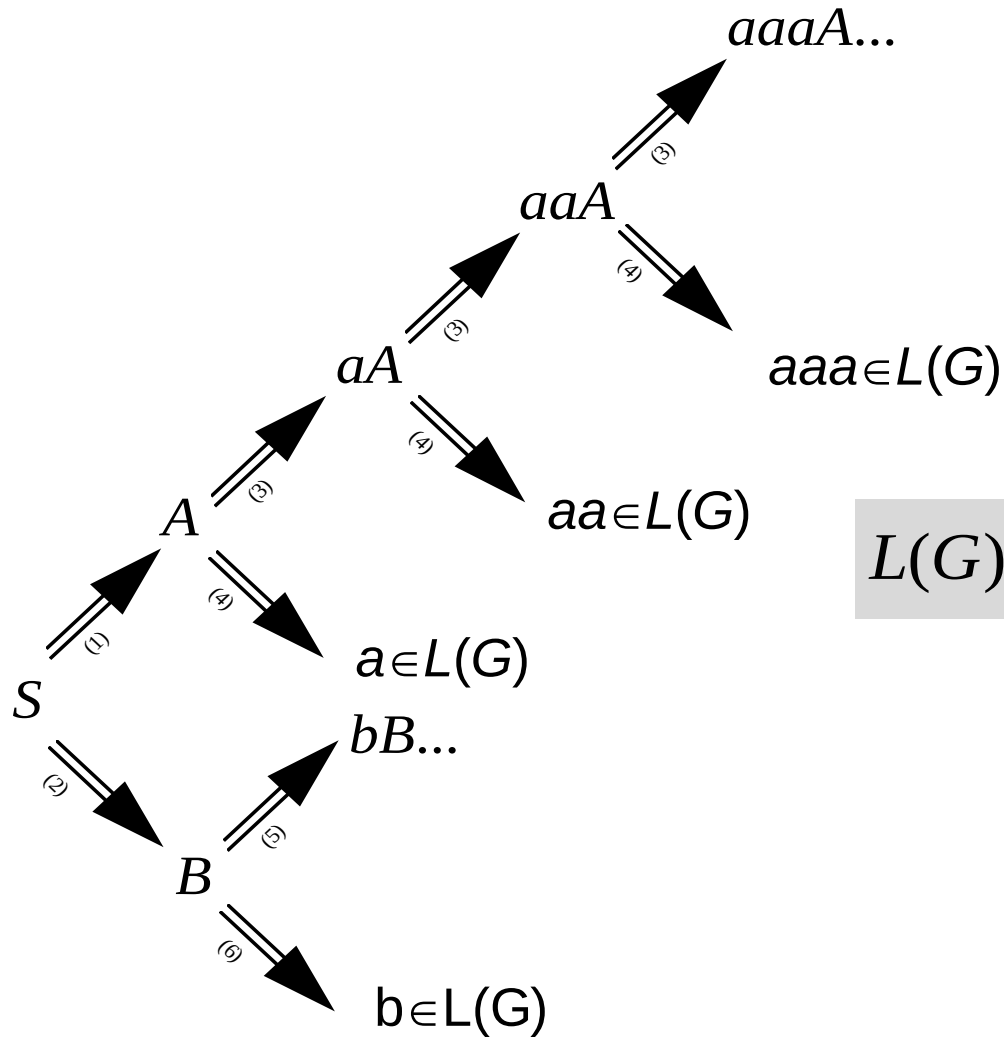
Esempio

$$S \xrightarrow{(1) \ (2)} A \mid B, \quad A \xrightarrow{(3) \ (4)} aA \mid a, \quad B \xrightarrow{(5) \ (6)} bB \mid b$$



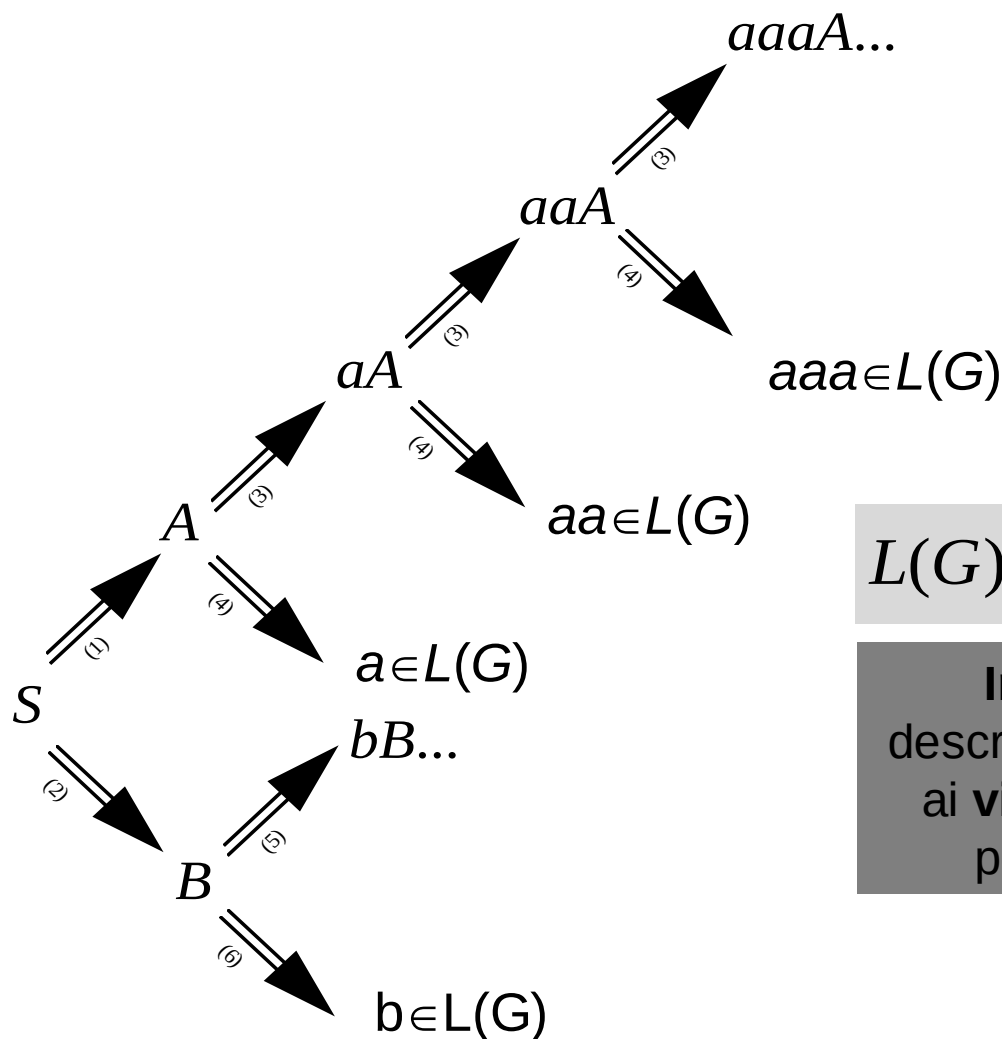
Definiamo il descrittore del linguaggio

Esempio



$$L(G) = \{a^n \mid n > 0\} \cup \{b^n \mid n > 0\}$$

Esempio



$$L(G) = \{a^n | n > 0\} \cup \{b^n | n > 0\}$$

Importante: nel definire il descrittore, fate sempre attenzione ai **vincoli sui simboli** (es. $n > 0$) perché sono fondamentali.

Esempio

- Sia data la seguente grammatica:

$$S \xrightarrow{(1)} A \mid \overset{(2)}{B}, \quad A \xrightarrow{(3)} aA \mid \overset{(4)}{a}, \quad B \xrightarrow{(5)} bB \mid \overset{(6)}{b}$$

Nota

Non sappiamo se applicare $S \xrightarrow{(1)} A$ oppure $S \xrightarrow{(2)} B$ inizialmente. **I meccanismi di costruzione di un linguaggio sono generalmente non deterministici**, poiché può non essere univoca la sostituzione da operare ad una forma di frase se uno stesso NT si trova a sinistra di 2 o più produzioni, come illustrato nella figura precedente.

Recap Conclusivo

- Dunque, una grammatica è uno **strumento generativo** di un linguaggio perché, data una qualsiasi parola di quel linguaggio, possiamo risalire mediante le produzioni al simbolo di partenza della grammatica.
 - **Punto di vista generativo!**

Recap Conclusivo

- Dunque, una grammatica è uno **strumento generativo** di un linguaggio perché, data una qualsiasi parola di quel linguaggio, possiamo risalire mediante le produzioni al simbolo di partenza della grammatica.
 - **Punto di vista generativo!**
- Viceversa, dato il simbolo di partenza di una grammatica, **seguendo uno qualsiasi dei cammini dell'albero di derivazione, si produce una parola “valida” del linguaggio.**
 - **Punto di vista riconoscitivo!**

Importantissimo

$$L \neq L(G)$$

Non fate confusione tra L ed $L(G)$

Non sono la stessa cosa

Importantissimo

$$L \neq L(G)$$

L è un linguaggio, cioè un sottoinsieme di X^* ricavato applicando delle regole (che non conosciamo)

L(G) è il linguaggio generato da una grammatica (che abbiamo progettato noi), cioè tutte le stringhe che si possono ottenere a partire dalle regole di G

Importantissimo

$$L \neq L(G)$$

Se la grammatica che abbiamo progettato è corretta, i due insieme sono equivalenti, ma non è automatico.

Come abbiamo visto nel caso della grammatica degli interi relativi, $L(G)$ non era identico a L perchè generava stringhe non corrette

L'obiettivo degli esercizi è farli corrispondere

Importantissimo

$$L = L(G)$$

Come facciamo a capire se una grammatica è corretta?

In generale, dato un linguaggio L ed una grammatica G , **non esiste un algoritmo in grado di dimostrare che la grammatica genera il linguaggio**, ossia che $L = L(G)$. Più specificamente, **non** esiste un algoritmo che stabilisce se una data stringa è generata o no dalla grammatica presa in considerazione.



Importantissimo

$$L = L(G)$$

Come facciamo a capire se una grammatica è corretta?

Il problema di dimostrare la correttezza di una grammatica **non è risolubile algebricamente, in generale.**

(Non è sorprendente, essendo L un insieme infinito)

Osservazione

- In molti casi importanti, però, è **possibile dimostrare per induzione** che una particolare grammatica genera proprio un particolare linguaggio ($L=L(G)$).
- Queste dimostrazioni ci consentono di stabilire se, data una grammatica G ed un linguaggio L , risulta:
 - $w \in L(G) \Rightarrow w \in L$ cioè $L(G) \subseteq L$
 - $w \in L \Rightarrow w \in L(G)$ cioè $L \subseteq L(G)$

Osservazione

- In molti casi importanti, però, è **possibile dimostrare per induzione** che una particolare grammatica genera proprio un particolare linguaggio ($L=L(G)$).
- Queste dimostrazioni ci consentono di stabilire se, data una grammatica G ed un linguaggio L , risulta:

□ $w \in L(G) \Rightarrow w \in L$ cioè $L(G) \subseteq L$

□ $w \in L \Rightarrow w \in L(G)$ cioè $L \subseteq L(G)$

La grammatica G genera solo stringhe appartenenti al linguaggio L .

Osservazione

- In molti casi importanti, però, è possibile dimostrare **per induzione** che una particolare grammatica genera proprio un particolare linguaggio. ($L=L(G)$).
- Queste dimostrazioni ci consentono di stabilire se, data una grammatica G ed un linguaggio L , risulta:

□ $w \in L(G) \Rightarrow w \in L$ cioè $L(G) \subseteq L$

□ $w \in L \Rightarrow w \in L(G)$ cioè $L \subseteq L(G)$

Il linguaggio L comprende solo parole generabili dalla grammatica G .

Principio di induzione

- Come vedremo nell'esercitazione, questo tipo di inclusione si dimostra mediante il principio di induzione
- Sia n_0 un intero e sia **$P=P(n)$ un enunciato** che ha senso per ogni intero maggiore o uguale ad n_0 .
- Se:
 - $P(n_0)$ è vero (**passo base**)
 - Per ogni $n > n_0$, $P(n-1)$ vero implica $P(n)$ vero allora $P(n)$ è vero per tutti gli n maggiori o uguali ad n_0 (**passo induttivo**)

Domande?



Linguaggi di Programmazione
docente: Cataldo Musto
cataldo.musto@uniba.it